

Regression with Multiple Input Variables

Multiple Features, Vectorization, and Gradient Descent in n Dimensions

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 2 · Part 1

Where we are, and where this goes

Week 1 gave us

$$f_{w,b}(x) = wx + b$$

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2$$

$$w := w - \alpha\frac{\partial J}{\partial w}, \quad b := b - \alpha\frac{\partial J}{\partial b}$$

One feature: floor area. But nobody prices a flat on area alone — bedrooms, age and location all matter.

Today

1. Multiple features and the notation for them
2. The model as a dot product
3. Vectorization, and why it is fast
4. Gradient descent for n features
5. The normal equation, in passing

Nothing conceptually new: the same three-part recipe, with w promoted from a number to a vector.

Multiple Features

One feature was never enough

Size x_1 (k sq ft)	Bedrooms x_2	Floors x_3	Age x_4 (yr)	Price y (lakh BDT)
2.10	5	1	45	46.0
1.42	3	2	40	33.2
1.53	3	2	30	31.5
0.85	2	1	36	17.8
1.94	4	2	15	42.0

Notation

- n — number of features (here $n = 4$).
- $x^{(i)}$ — the i-th training example, now a *vector*.
- $x_j^{(i)}$ — feature j of example i.

From the table:

$$x^{(2)} = \begin{bmatrix} 1.42 & 3 & 2 & 40 \end{bmatrix}, \qquad x_3^{(2)} = 2.$$

$x^{(i)}$ is a row of the table; $x_j^{(i)}$ is one cell.

The model with n features

Previously $f_{w,b}(x) = wx + b$. Now every feature carries its own weight:

$$f_{\mathbf{w},b}(\mathbf{x}) \;=\; w_1x_1 + w_2x_2 + \cdots + w_nx_n + b \;=\; \mathbf{w}\cdot\mathbf{x} + b,$$

writing $\mathbf{w} = [w_1, \ldots, w_n]^\top$, $\mathbf{x} = [x_1, \ldots, x_n]^\top$ and $\mathbf{w}\cdot\mathbf{x} = \sum_{j=1}^{n} w_j x_j$ for the dot product. The bias b stays a scalar.

A plausible fitted model for our flats:

$$f(\mathbf{x}) = 0.1x_1 + 4x_2 + 10x_3 - 2x_4 + 80$$

Every parameter reads as a price: $b = 80$ is a base in lakh, $w_2 = 4$ says an extra bedroom adds 4 lakh, $w_4 = -2$ says each year of age costs 2 lakh.

Key idea

This is **multiple** linear regression — several *inputs*. It is not "multivariate regression", which means several *outputs*.

Vectorization

Three ways to compute the same thing

With $n = 4$, $\mathbf{w} = [w_1, \ldots, w_4]$ and $\mathbf{x} = [x_1, \ldots, x_4]$:

1. Written out

```
f = w[0]*x[0] + w[1]*x[1] + w[2]*x[2] + w[3]*x[3] + b
```

Fine for $n = 4$; unwritable for $n = 100$.

2. A loop

```
f = 0
for j in range(n):  f = f + w[j] * x[j]
f = f + b
```

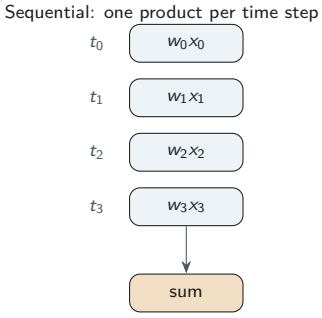
Scales in source length, but runs in the Python interpreter.

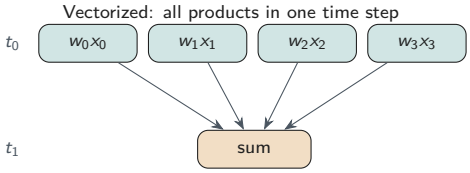
3. Vectorized

```
f = np.dot(w, x) + b
```

Shorter *and* far faster — the only version you should write.

Why vectorization is faster





`np.dot` hands the arrays to compiled code using the CPU's **SIMD** instructions — one instruction acting on several numbers at once — then a parallel tree reduction for the sum. On a GPU the same idea spans thousands of cores.

Key idea

The speed-up grows with n and m, so vectorization is a correctness-level concern, not a style preference.

Vectorizing the gradient descent step

Suppose $n = 16$ and we have all sixteen derivatives $d_1, \ldots, d_{16}$.

Without vectorization

```
for j in range(16):
    w[j] = w[j] - 0.1 * d[j]
```

Sixteen steps, executed one after another.

With vectorization

```
w = w - 0.1 * d
```

One step. NumPy updates all sixteen weights in parallel.

Common pitfall

`w * x` is *elementwise* and returns a vector; `np.dot(w, x)` sums those products and returns the scalar the model needs. Confusing them yields an array of the wrong shape, often with no error until much later.

Gradient Descent for Multiple Features

The cost, and the gradient

With $\mathbf{w} \in \mathbb{R}^n$ and $b \in \mathbb{R}$, the cost is unchanged in form:

$$J(\mathbf{w}, b) \;=\; \frac{1}{2m} \sum_{i=1}^{m} \big(f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}\big)^2, \qquad f_{\mathbf{w},b}(x^{(i)}) = \mathbf{w} \cdot x^{(i)} + b.$$

Differentiating with respect to one weight w_j — everything else is a constant:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \big(f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}\big)\, x_j^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \big(f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}\big).$$

Key idea

Compare with Week 1: $\partial J/\partial w = \frac{1}{m} \sum (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$. The only change is that $x^{(i)}$ became $x_j^{(i)}$ — feature j of example i. The residual is shared by every parameter; only the weighting differs.

Gradient descent, n features

Repeat until convergence, updating all $n+1$ parameters *simultaneously*:

$$w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \big(f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}\big) x_j^{(i)} \qquad \text{for } j = 1, \ldots, n$$

$$b := b - \alpha \frac{1}{m} \sum_{i=1}^{m} \big(f_{\mathbf{w},b}(x^{(i)}) - y^{(i)}\big)$$

In vector form the whole update is two lines:

$$\mathbf{w} := \mathbf{w} - \alpha \nabla_{\mathbf{w}} J, \qquad b := b - \alpha \frac{\partial J}{\partial b}, \qquad \nabla_{\mathbf{w}} J = \frac{1}{m} X^{\top}(X\mathbf{w} + b\mathbf{1} - \mathbf{y}).$$

Common pitfall

"Simultaneously" now means all $n+1$ parameters. Compute the entire gradient from the old $(\mathbf{w}, b)$, then assign. In NumPy: build `dj_dw` and `dj_db` first, and only then write `w -= alpha*dj_dw; b -= alpha*dj_db`.

Computing the gradient for all n features

```
def compute_gradient(X, y, w, b):
    m = X.shape[0]
    err = (X @ w + b) - y          # shape (m,)
    dj_dw = (X.T @ err) / m          # shape (n,)
    dj_db = np.sum(err) / m          # scalar
    return dj_dw, dj_db
```

Shapes are the contract

- `X` is (m, n), `y` is $(m,)$
- `w` is $(n,)$, `b` is a scalar
- `X @ w` is $(m,)$ — one prediction per example
- `X.T @ err` is $(n,)$ — one derivative per feature

Why `X.T @ err` is the gradient

Row j of $X^\top$ is feature j across all examples. Dotting it with the residual vector gives $\sum_i (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$ — exactly $m\,\partial J/\partial w_j$.

An Alternative: the Normal Equation

Solving for w in one step

Linear regression is unusual: its minimum has a closed form. Setting $\nabla J = \mathbf{0}$ and solving gives the **normal equation**

$$\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$$

where X carries a column of ones to absorb b.

In its favour

- No α to choose.
- No iterating, no convergence check.
- Exact, up to floating point.

Against

- Only for *linear* regression — no logistic regression, no neural nets.
- Needs $(X^\top X)^{-1}$: cost grows like n^3, so it is unusable past $n \sim 10^4$.
- Fails when $X^\top X$ is singular (duplicated or redundant features).

Key idea

Use it as a *test oracle*: on a small problem, check that your gradient descent converges to what `np.linalg.lstsq` returns. Gradient descent remains the method that generalises to everything else in this course.

Wrap-up and Assessment

Summary of Part 1

Concepts

- n features; $x^{(i)}$ is a vector, $x_j^{(i)}$ one of its entries.
- $f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$.
- Vectorization is parallel hardware, not tidier code.
- The gradient's form is unchanged; only the weighting index moves.
- The normal equation solves this one model exactly, and nothing else.

Formulas

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\mathbf{w},b}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

$$\mathbf{w} := \mathbf{w} - \alpha \nabla_{\mathbf{w}} J$$

Next

Part 2: why unscaled features cripple gradient descent, how to check convergence, choosing α, and building new features.

Descriptive questions

1. Define n, m, $x^{(i)}$ and $x_j^{(i)}$. For the flats table on slide 4, state $x^{(4)}$, $x_2^{(4)}$ and n. **[5]**
2. Write the multiple linear regression model in both expanded and vector form, and explain why b is not part of $\mathbf{w}$. **[4]**
3. Distinguish *multiple* linear regression from *multivariate* regression. **[3]**
4. Derive $\partial J/\partial w_j$ for the multi-feature squared-error cost. State precisely where it differs from the single-feature case and why. **[8]**
5. Explain what vectorization means and why `np.dot(w,x)` outperforms an equivalent Python loop. Refer to SIMD in your answer. **[6]**

Descriptive questions (continued)

6. For $\mathbf{w} = [0.1, 4, 10, -2]$ and $b = 80$, compute the predicted price of a flat with $\mathbf{x} = [1.5, 3, 2, 20]$. Interpret each of the four terms in words. **[6]**
7. Given `X` of shape (m, n), `w` of shape $(n,)$ and `y` of shape $(m,)$, state the shape of `X @ w`, `X.T @ err` and `np.sum(err)`. Explain why `X.T @ err` is the weight gradient. **[6]**
8. State the normal equation and give two reasons it does not replace gradient descent in this course. **[5]**
9. A student writes `f = w * x + b` instead of `np.dot(w,x) + b`. Describe what is actually computed, its shape, and why the bug may go unnoticed. **[5]**

Design questions

1. **Design the feature set.** You must predict monthly electricity bills for households in Chattogram. Propose five features, giving units for each, and state the sign you expect each weight to take and why. Identify one feature that looks useful but would leak the answer. **[12]**
2. **Design for interpretability.** A bank will use your linear model to explain loan decisions to customers. Which choices in the model — feature units, feature selection, the presence of b — affect whether w_j can be quoted as a reason? Write the sentence you would put in front of a customer for one feature. **[10]**
3. **Design a shape-check harness.** Write three assertions that `compute_gradient(X, y, w, b)` should satisfy for any valid input, such that a transposed matrix or a misused $*$ is caught immediately. State what each assertion rules out. **[10]**
4. **Choose the solver.** For each case, argue for gradient descent or the normal equation: (a) $m = 500$, $n = 6$; (b) $m = 10^7$, $n = 3000$; (c) $n = 50$ where two features are duplicates; (d) a logistic regression on $m = 2000$, $n = 10$. **[12]**

Reading

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§3.2** — multiple linear regression: estimating the coefficients, interpreting them, and what the individual w_j do and do not mean.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§3.1–3.1.1** — linear basis function models and the normal equations in matrix form.

Lab (CSE 816, Week 2): NumPy vectorization; multiple linear regression.

Practice quiz: *Multiple linear regression*.